



Islamic University of Technology

Department of Computer Science and Engineering

Orthogonal Defect Classification on Defects4J

Using an LLM-Driven Scientific Approach

Thesis Defence · B.Sc. in Computer Science and Engineering

Presented by

Md. Sakib Hossain (210042133)

Mohammad Nahiyen Kabir (210042168)

Hasin Mahtab Alvee (210042174)

Supervisor

Lutfun Nahar Lota, Assistant Professor

Co-supervisor

Ishmam Tashdeed, Lecturer

Outline



1

Context

slides 3–8

- Motivation and Defects4J
- ODC and related work
- Pre-fix vs post-fix
- Research questions

2

Methodology

slides 9–16

- Pipeline and evidence
- Conditions and prompts
- The reasoning loop
- Evaluation and setup

3

Results

slides 17–24

- RQ1–RQ2: types, coverage
- RQ3: agreement and drift
- RQ4: taxonomy and loop
- Threats and conclusion



Motivation: Classifying Defects in Defects4J

836

reproducible real bugs

17

Java projects [5]

- Defect types need different debugging strategies
- Aggregated labels expose recurring mistakes
- Manual triage is accurate, but does not scale

WHAT EVERY DEFECTS4J BUG SHIPS WITH

Bug report

Failing tests + stack traces

Buggy source

Fixed source

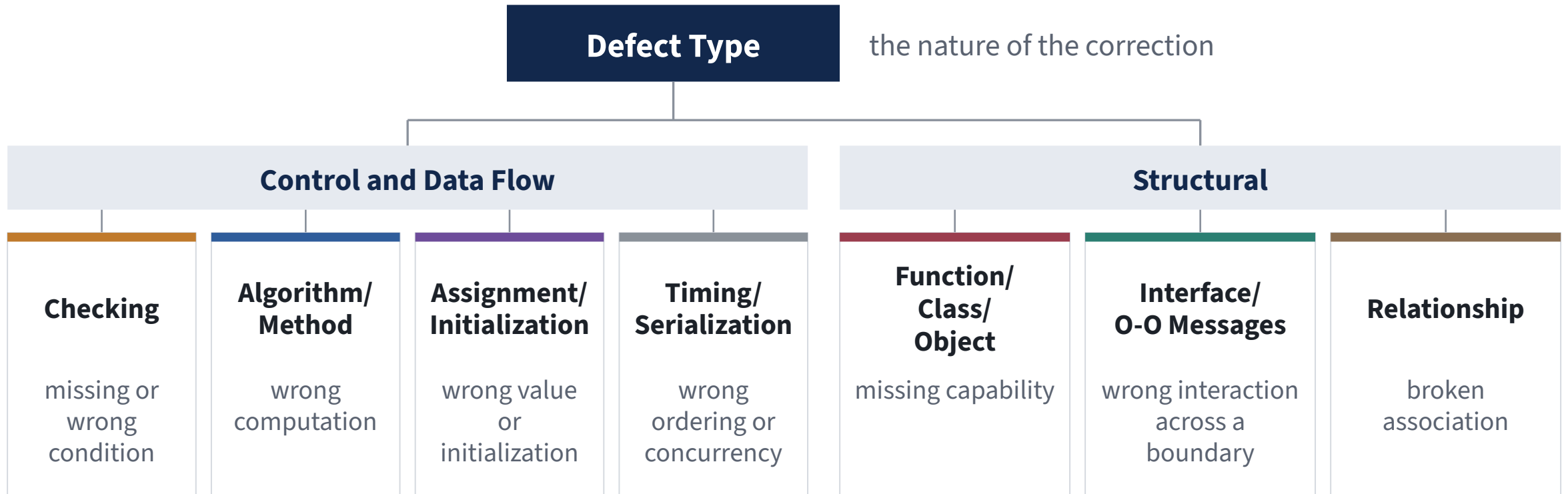
Fix diff

Every bug ships with its fix, so one bug can be classified **before** and **after** it.



Orthogonal Defect Classification (ODC)

ODC [4, 15] describes a defect when it is found (*opener*) and when it is fixed (*closer*). We classify one closer attribute:

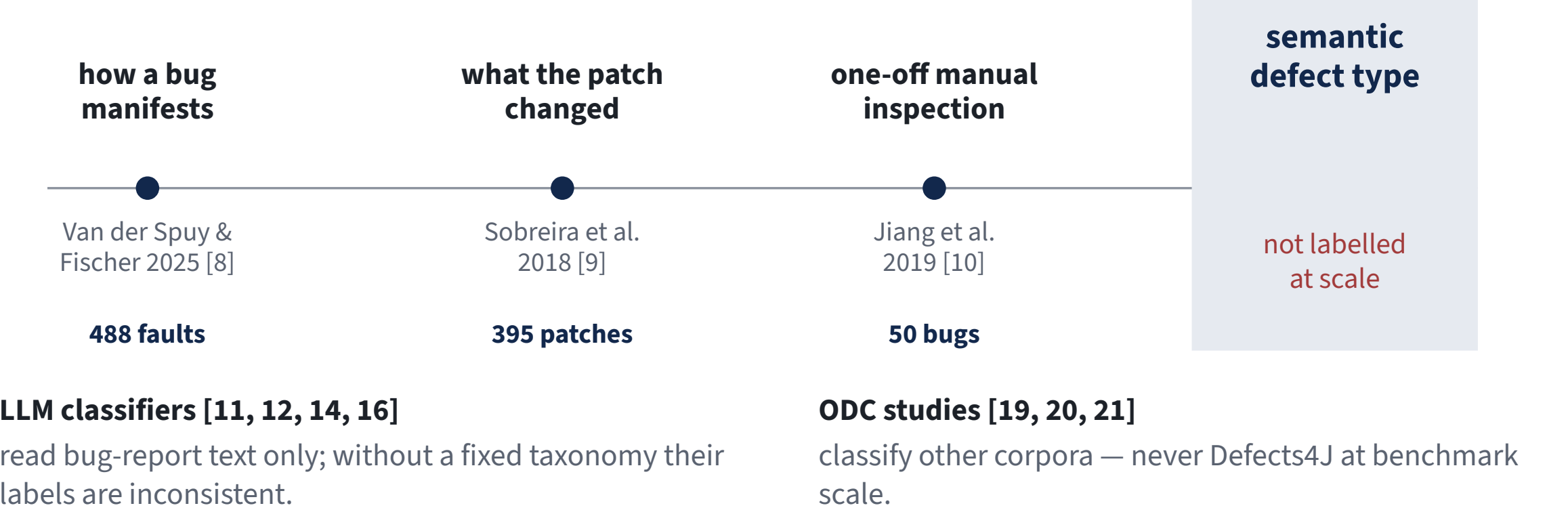


+ Other — escape category added in this study, allowed only with a written justification



Related Work and the Research Gap

WHAT PRIOR WORK ON DEFECTS4J ACTUALLY LABELS



Gap: no semantic defect-type labels for Defects4J, and no way to evaluate them without ground truth.



Problem Statement

Classify Defects4J bugs into ODC defect types **from pre-fix evidence alone** — and evaluate that classification **without ground-truth labels**.

A SOLUTION THEREFORE HAS TO BE

Context-aware

code, tests, traces and reports together

Semantic

the defect mechanism, not the syntax

Taxonomy-constrained

a fixed, comparable label space

Evaluable

carries its own reference, without ground truth



Core Idea: One Bug, Two Classifications

DEFECTS4J CHART_9

```
org.jfree.data.time.TimeSeries
```

```
    endIndex = endIndex - 1;
```

```
- if (endIndex < 0) {
```

```
+ if ((endIndex < 0) || (endIndex < startIndex)) {
```

```
    emptyRange = true;
```

```
}
```

The fix widens a guard to catch an empty range.

Pre-fix

report · tests · trace · code

Triage: no fix exists yet.

Post-fix

the same evidence + the fix diff

Informed by the correction.

DEFECTS4J HAS NO ODC LABELS, SO WE DEFINE

Post-fix label

= the fix-informed reference

Pre-fix vs post-fix

= does the triage label hold?

Labels differ

= drift

Agreement is not correctness: we measure how closely a pre-fix label reproduces a better-informed reference.



Research Questions

RQ1

Distribution

Which ODC types dominate, and does the mix differ by project?

RQ2

Coverage

Do the seven types cover every bug, or is “Other” needed?

RQ3

Accuracy

How often does the pre-fix label match the post-fix reference?

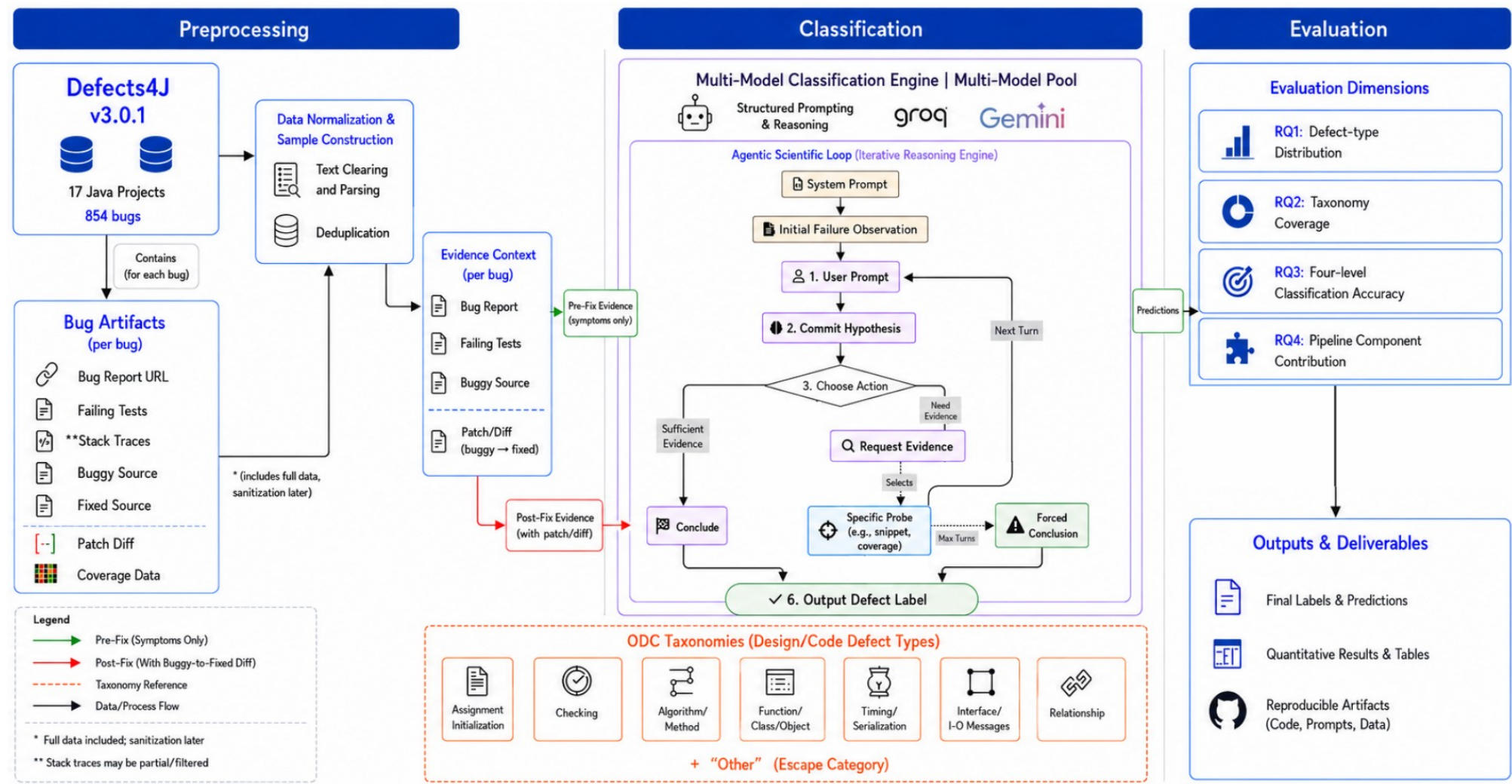
RQ4

Components

What do (a) the taxonomy and (b) the enforced loop contribute?

410 bugs · six Defects4J projects · 1,640 classifications.

Pipeline Overview





Evidence Collection: the Case File

CONTEXT.JSON • ONE PER BUG AND EVIDENCE MODE

```
{
  "bug_report":      sanitized,
  "failing_tests":   name, message,
                    stack trace,
  "suspicious_frames": project frames,
  "code_snippets":   ±12 / ±18 lines,
  "coverage":        line, branch,
  "hidden_oracles":  modified classes,
  "fix_diff":         post-fix only
}
```

Suspicious frames

Only frames in the project's own source; JUnit, JDK and build-tool frames are dropped.

Sanitized pre-fix evidence

Classes touched by the fix and “fixed in” discussion are removed.

Hidden oracle

Modified classes are kept for evaluation, never put in the prompt.

Pre-fix and post-fix runs share one evidence store; only the fix diff differs.



Classification Conditions

A condition = taxonomy (label space) × strategy (reasoning).

STRATEGY	TAXONOMY		
	free	closed	open
zero	zero-free	—	—
few	—	few-closed	few-open
scientific	—	scientific-closed	scientific-open default

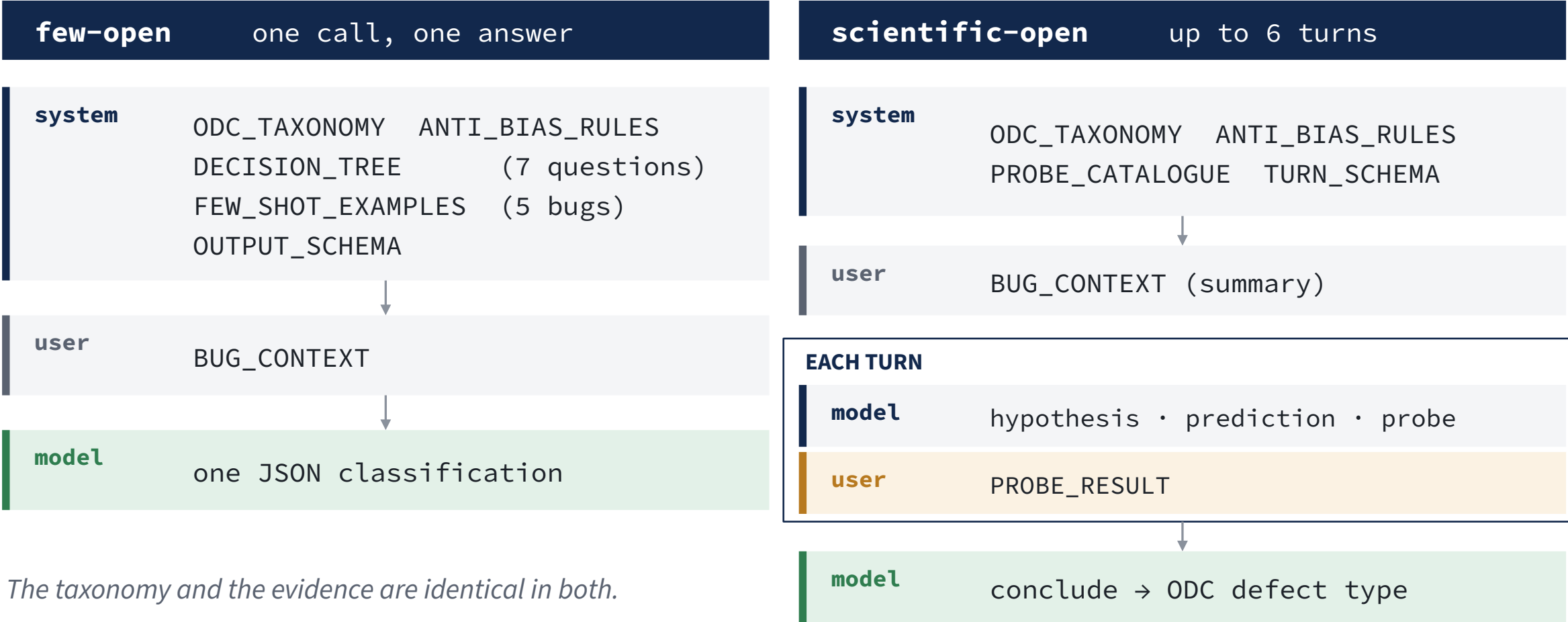
Five of nine combinations are valid: zero uses no taxonomy, and free defines none.

TAXONOMY	
free	the model's own words
closed	one of the 7 ODC types
open	7 types + a justified “Other”
STRATEGY	
zero	one plain prompt
few	definitions, tree, 5 examples
scientific	enforced hypothesis–probe loop



Prompt Design

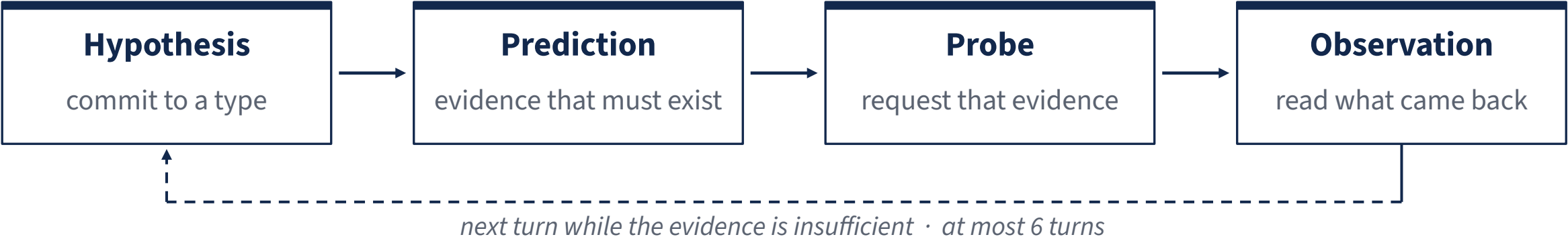
Capitalised names are prompt components: defined once, or filled in per bug.



The taxonomy and the evidence are identical in both.



The Enforced Scientific Reasoning Loop



Adapted from AutoSD [17, 18]: from locating a fault to naming its type.

Conclude: ODC label

FIVE PROBES, EXECUTED AGAINST THE EVIDENCE STORE

list_evidence()	what evidence exists	full_stack_trace(test)	trace and message
snippet(class)	source snippets	coverage(class)	line and branch
bug_report()	the full report		

The prediction is fixed before the evidence arrives, so a first guess cannot be rationalised.



Evaluation Framework: Four Tiers of Agreement

TIERS 1–3 ARE NESTED: STRICT INSIDE FORGIVING

Tier 3 Family match

both types in the same family

Tier 2 Top-2 match

either label is the other's alternative

Tier 1 Strict match

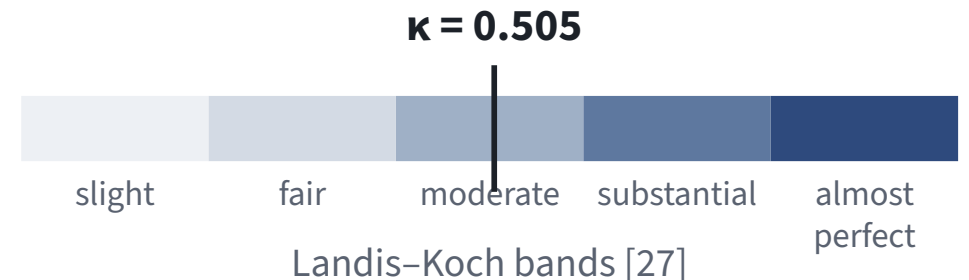
the same ODC type

Three types cover 97.8% of labels, so shuffled labels still reach >92% on Tiers 2–3: we use them to grade disagreements, not as scores.

TIER 4 • COHEN'S KAPPA

$$\kappa = (p_o - p_e) / (1 - p_e)$$

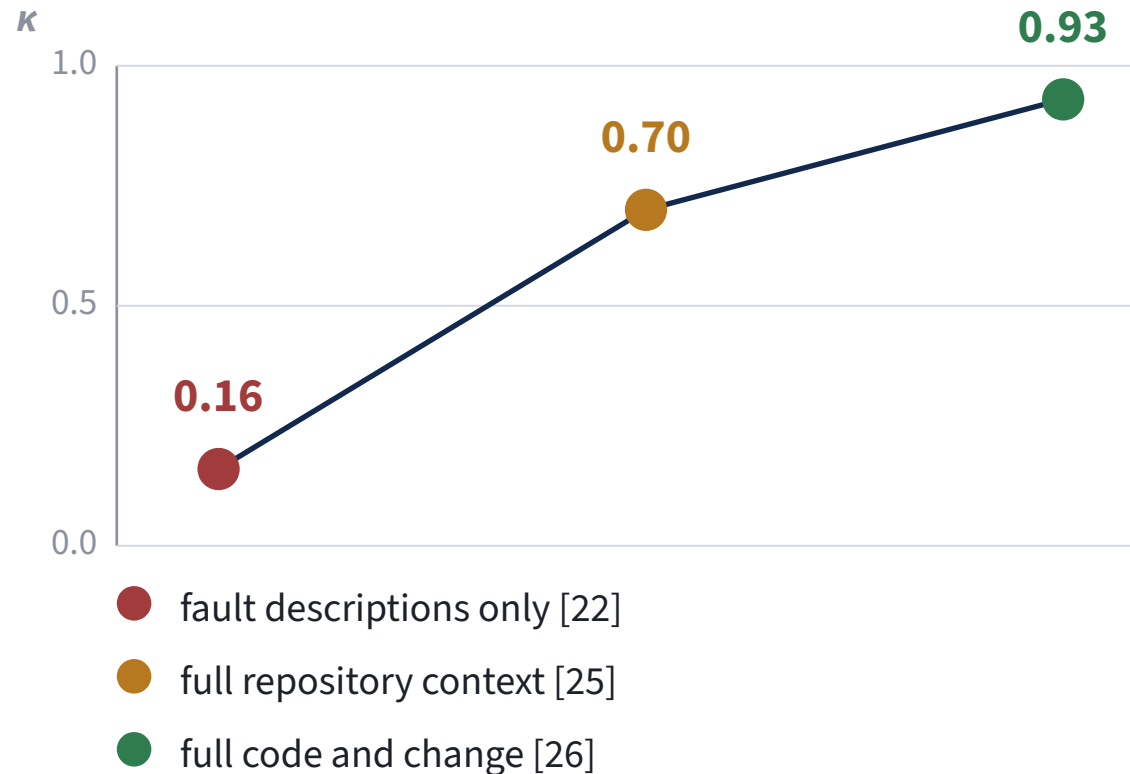
p_o is observed agreement, p_e the agreement expected by chance.





Why the Post-Fix Label Is a Valid Reference

HUMAN AGREEMENT ON ODC-STYLE CLASSIFICATION RISES WITH EVIDENCE



OUR DATA BEHAVE THE SAME WAY

68.0% → 76.3%

strategy agreement, without → with the fix

6 of 6

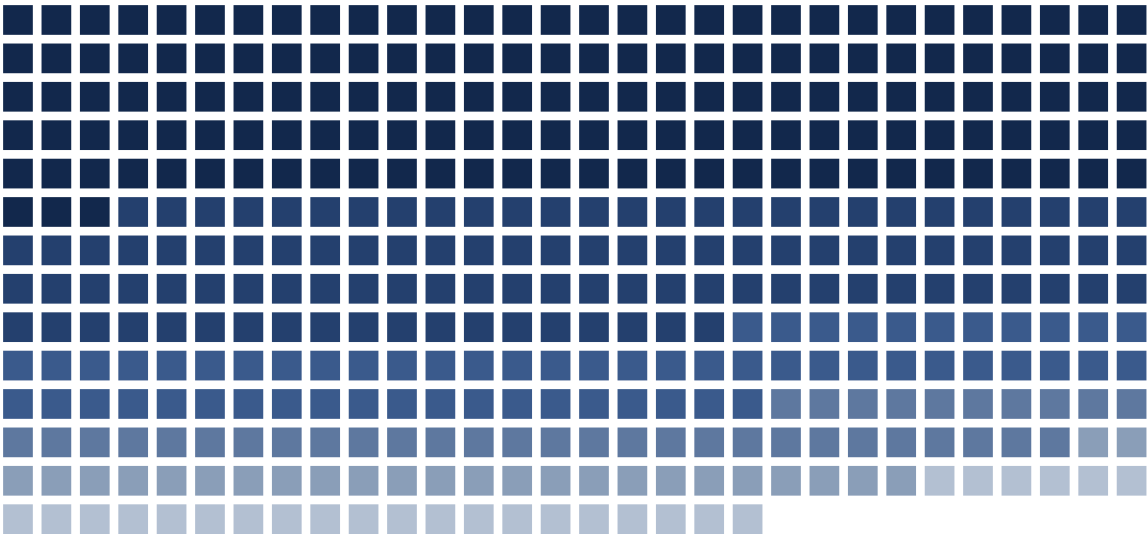
projects agree more once the fix is visible

The run that sees the fix is better informed — but two labels can agree and both be wrong.



Experimental Setup

410 BUGS • ONE SQUARE PER BUG



- Closure 153
- Math 106
- Lang 61
- Mockito 38
- Chart 26
- Time 26

Reporting follows the guidelines for empirical studies with LLMs [28].

MODELS

gemi-3.1-flash-lite-preview	410 bugs
gpt-5-mini	13 bugs

EVERY BUG, FOUR TIMES

2 conditions

scientific-open and few-open

2 evidence modes

pre-fix and post-fix

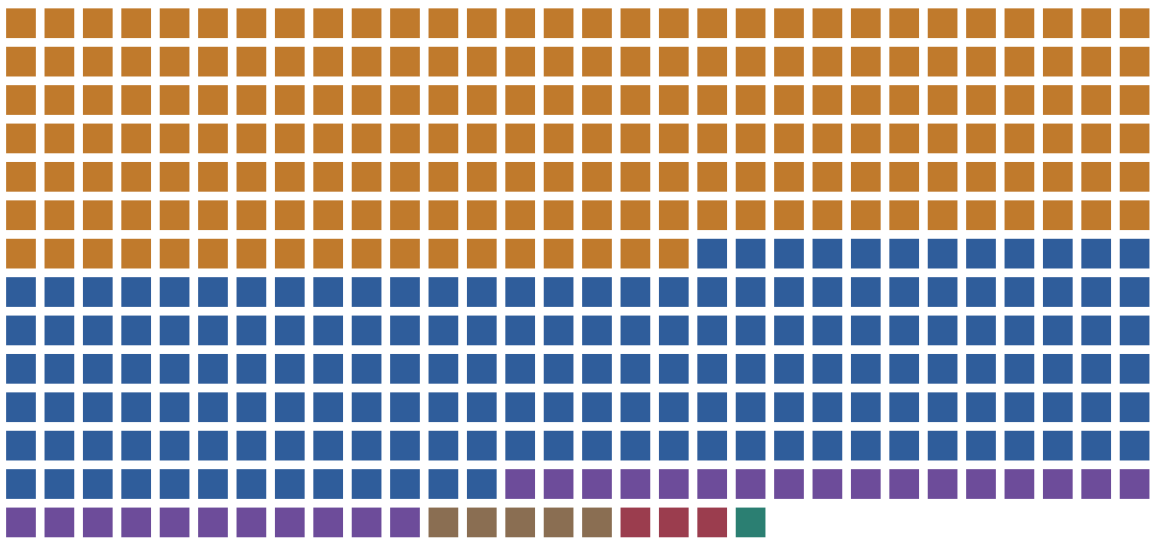
1,640

classifications, plus a zero-free baseline



RQ1–RQ2: Distribution and Coverage

RQ1 • 410 PRE-FIX LABELS, ONE SQUARE PER BUG



97.8% in three Control and Data Flow types; Timing never occurs. Mix by project: scientific **p = 0.096**, few-shot **p = 0.010**.

RQ2 • THE “OTHER” CATEGORY

0 of 1,640

classifications chose “Other”, although it was offered every time.

≤ 0.2%

95% upper bound on the escape rate

6 of 7

ODC types actually used



RQ3: Agreement with the Fix-Informed Reference

70.2%

of pre-fix labels match the post-fix reference exactly

95% CI [65.6, 74.9]

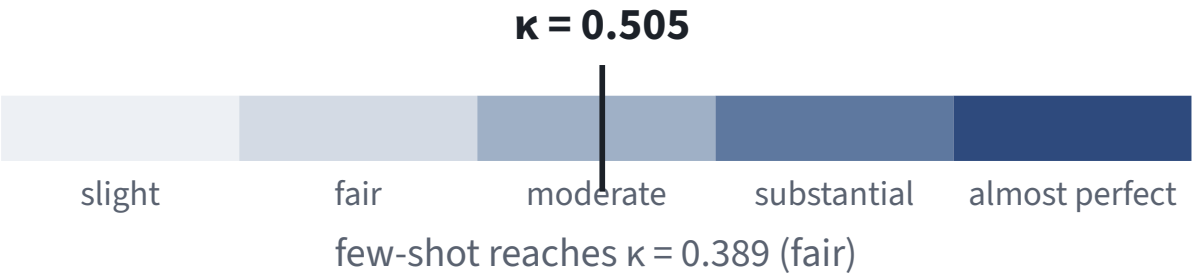
ALL 410 BUGS (SCIENTIFIC-OPEN)



near miss: one label is the other's alternative, or both share a family

unrelated 2.4%

CHANCE-CORRECTED AGREEMENT



WHERE THE DISAGREEMENT COMES FROM

Two fix-aware classifications agree only **76.3%** of the time. Pre-fix sits about 6 points below that ceiling; the rest is ODC boundary ambiguity.

93.2%

name the reference type as the label or as a recorded alternative



RQ3: How Labels Change When the Fix Is Visible

PRE-FIX LABEL (ROW) → POST-FIX LABEL (COLUMN)

	Checking	Algorithm	Assignment	other types
Checking	148	37	8	5
Algorithm	38	119	10	8
Assignment	3	2	18	5
other types	3	2	1	3

outlined the label did not change **darker** more bugs

The two dark cells cancel out: 38 bugs move one way, 37 the other.

Algorithm/Method vs Checking is also the boundary that human raters confuse [22].

29.8%

of bugs change label (122 of 410)

80.3%

of changes stay in these three types

0.051

total variation distance: the mix barely moves

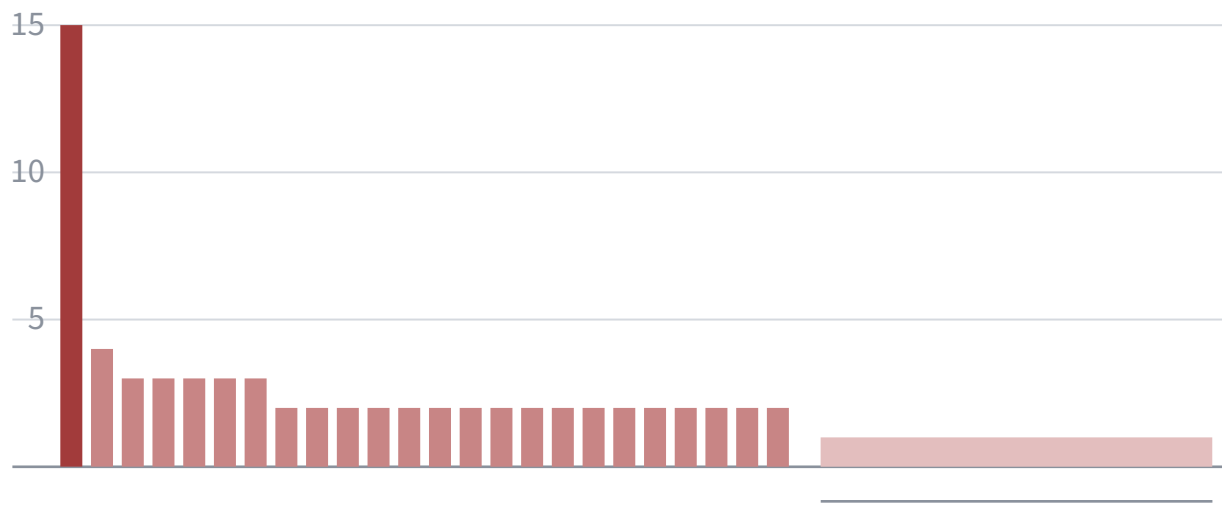


RQ4a: Contribution of the ODC Taxonomy

366 different labels

when the model writes its own (zero-free, 410 bugs)

“Null Pointer Dereference” 15 bugs



342 labels used exactly once

The same bug (Chart_9), written twice:
“Improper input validation logic”
“Incorrect boundary condition logic”

6 comparable labels

with the ODC taxonomy (scientific-open)



every bug lands in one of six buckets that can be counted

Condition	Same label, both modes
zero-free	15.1%
few-open	66.6%
scientific-open	70.2%

A fixed taxonomy lets us count anything at all.



RQ4b: Contribution of the Scientific Loop

$\Delta\kappa = +0.115$

κ 0.389 $\rightarrow$ 0.505

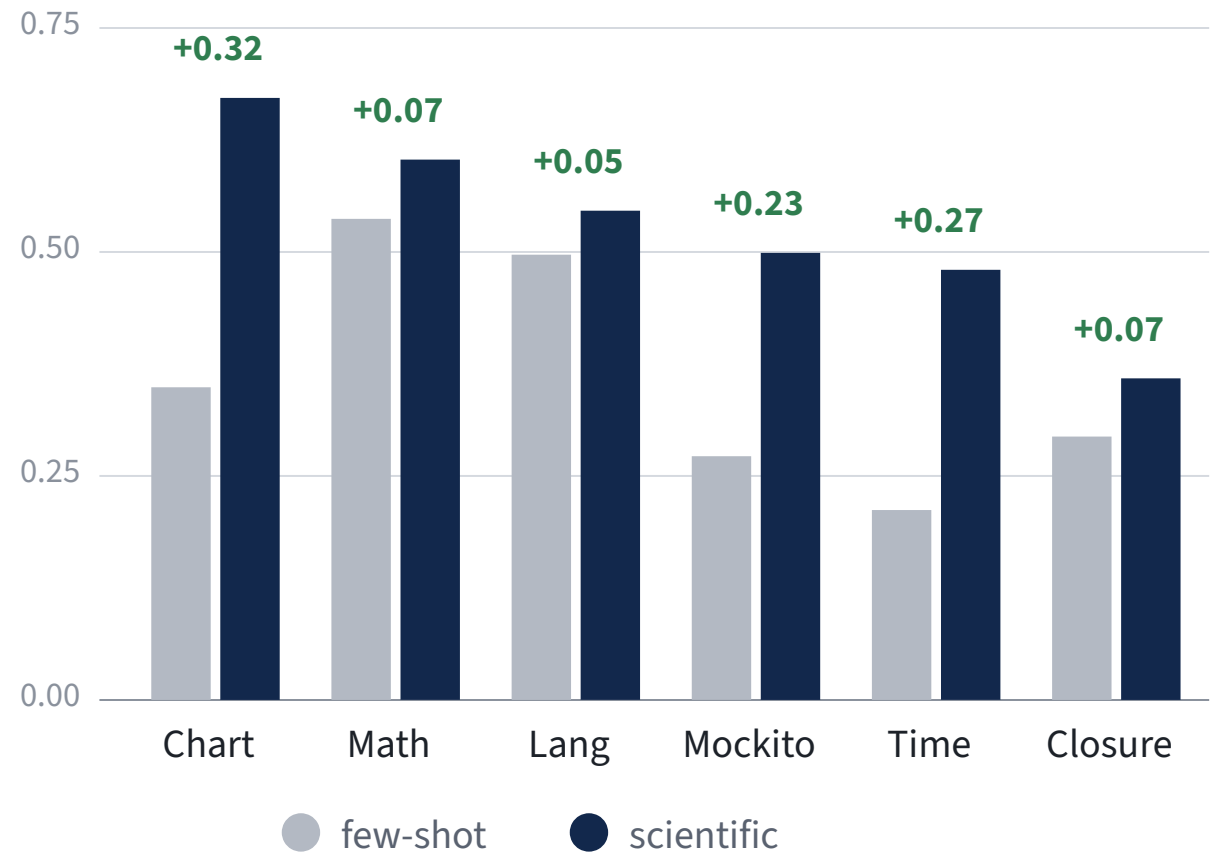
Statistically significant:
95% CI [0.026, 0.208], $p \approx 0.012$ (paired bootstrap)

6 of 6 projects improve; sign test $p = 0.016$

$p = 0.221$ McNemar on raw agreement: few-shot's 265 Algorithm labels inflate chance agreement

Closure smallest gain (+0.065); its interval still includes 0

PER-PROJECT K · EVERY PROJECT IMPROVES





Qualitative Analysis of Thirteen Bugs

FAILURE MODES WE FOUND

Symptom-site bias the label follows where the exception surfaced	Chart_17
Bug-report anchoring a fix suggested in the report becomes the label	Math_90
Surface syntax the post-fix label follows the patch shape	Math_23
Underdetermined two valid fixes imply two ODC types	Lang_20

WHERE THE DEVELOPER HAD A CHOICE

Math_90	could have been a Comparable check (CHK) developer chose a typed overload (INT)
Math_17	could have been a range check (CHK) developer chose both: check + fallback
Time_3	could have been better arithmetic (ALG) developer chose amount != 0 guards (CHK)
Lang_20	could have been a null check (CHK) developer chose a new capacity (ASN)

It records the repair chosen — for **77%** of changed labels that type is already a recorded alternative.



Threats to Validity

Internal	non-deterministic output; evidence sanitized against fix leakage
Construct	agreement is a proxy; the modified class is absent for 285 of 410 bugs
External	6 of 17 projects; corpus results from one model; Java only
Conclusion	one run per condition; resampled values are estimates; a single rater for 13 bugs
Other	possible pre-training exposure; confidence fields are not a usable signal

Each threat is stated in the book together with the mitigation applied.



Conclusion and Future Work

97.8%

three types (RQ1)

0 of 1,640

“Other” (RQ2)

70.2%

pre-fix match (RQ3)

+0.115

$\Delta\kappa$ from the loop (RQ4)

CONTRIBUTIONS

- A taxonomy $\times$ strategy condition space
- An enforced loop with auditable transcripts
- Evaluation without ground truth
- 410 bugs, analysed with two LLMs

FUTURE WORK

- Human reference labels for a sample
- Grade the reasoning, not only the label
- Full corpus, more models, consensus
- Impact, Trigger and Age attributes



References (1/3)

- [1] G. Catolino et al., “Not All Bugs Are the Same: Understanding, Characterizing, and Classifying Bug Types,” *Journal of Systems and Software*, vol. 152, 2019.
- [2] T. Hirsch and B. Hofer, “Using Textual Bug Reports to Predict the Fault Category of Software Bugs,” *Array*, vol. 15, 2022.
- [3] R. Andrade et al., “An Empirical Study on the Classification of Bug Reports with Machine Learning,” *arXiv preprint*, 2025.
- [4] R. Chillarege et al., “Orthogonal Defect Classification – A Concept for In-Process Measurements,” *IEEE Transactions on Software Engineering*, vol. 18, no. 11, 1992.
- [5] R. Just, D. Jalali, and M. D. Ernst, “Defects4J: A Database of Existing Faults to Enable Controlled Testing Studies for Java Programs,” *Proc. International Symposium on Software Testing and Analysis (ISSTA)*, 2014.
- [6] M. N. Rafi et al., “Revisiting Defects4J for Fault Localization in Diverse Development Scenarios,” *IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*, 2025.
- [7] M. Martinez et al., “Automatic Repair of Real Bugs in Java: A Large-Scale Experiment on the Defects4J Dataset,” *Empirical Software Engineering*, vol. 22, no. 4, 2017.
- [8] A. Van der Spuy and B. Fischer, “An Anatomy of 488 Faults from Defects4J Based on the Control- and Data-Flow Graph Representations of Programs,” *Proc. 29th International Conference on Evaluation and Assessment in Software Engineering (EASE)*, 2025.
- [9] V. Sobreira et al., “Dissection of a Bug Dataset: Anatomy of 395 Patches from Defects4J,” *IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, 2018.
- [10] J. Jiang, Y. Xiong, and X. Xia, “A Manual Inspection of Defects4J Bugs and Its Implications for Automatic Program Repair,” *Science China Information Sciences*, vol. 62, no. 10, 2019.

References (2/3)



- [11] G. Colavito et al., “Large Language Models for Issue Report Classification,” Ital-IA 2024 Thematic Workshops, CEUR Workshop Proceedings, vol. 3762, 2024.
- [12] A. Koyuncu, “Exploring Fine-Grained Bug Report Categorization with Large Language Models and Prompt Engineering: An Empirical Study,” ACM Transactions on Software Engineering and Methodology, vol. 35, no. 3, 2026.
- [13] Y. Nong et al., “Chain-of-Thought Prompting of Large Language Models for Discovering and Fixing Software Vulnerabilities,” arXiv preprint, 2024.
- [14] C. Li et al., “KnowBug: Enhancing Large Language Models with Bug Report Knowledge for Deep Learning Framework Bug Prediction,” Knowledge-Based Systems, vol. 305, 2024.
- [15] IBM, “Orthogonal Defect Classification v5.2 for Software Design and Code,” IBM Research, 2013.
- [16] X. Du et al., “LLM-BRC: A Large Language Model-Based Bug Report Classification Framework,” Software Quality Journal, vol. 32, no. 3, 2024.
- [17] S. Kang et al., “Explainable Automated Debugging via Large Language Model-driven Scientific Debugging,” Empirical Software Engineering, vol. 30, no. 2, 2025.
- [18] S. Kang, G. An, and S. Yoo, “A Quantitative and Qualitative Evaluation of LLM-Based Explainable Fault Localization,” Proc. ACM on Software Engineering, vol. 1, no. FSE, 2024.
- [19] L. Huang et al., “AutoODC: Automated Generation of Orthogonal Defect Classifications,” Automated Software Engineering, vol. 22, no. 1, 2015.

References (3/3)



- [20] F. Thung, D. Lo, and L. Jiang, “Automatic Defect Categorization,” Proc. 19th Working Conference on Reverse Engineering (WCRE), 2012.
- [21] A. M. Esfahani, N. Kahani, and S. A. Ajila, “Understanding Defects in Generated Codes by Language Models,” 34th International Conference on Collaborative Advances in Software and Computing (CASCON), 2024.
- [22] K. Henningsson and C. Wohlin, “Assuring Fault Classification Agreement – An Empirical Evaluation,” Proc. 2004 International Symposium on Empirical Software Engineering (ISESE), 2004.
- [23] J. Hernández-González et al., “Learning to Classify Software Defects from Crowds: A Novel Approach,” Applied Soft Computing, vol. 62, 2018.
- [24] K. El Emam and I. Wieczorek, “The Repeatability of Code Defect Classifications,” Proc. Ninth International Symposium on Software Reliability Engineering (ISSRE), 1998.
- [25] A. Rahman and E. Farhana, “An Exploratory Characterization of Bugs in COVID-19 Software Projects,” arXiv preprint, 2020.
- [26] J. Agnelo, N. Laranjeiro, and J. Bernardino, “Using Orthogonal Defect Classification to Characterize NoSQL Database Defects,” Journal of Systems and Software, vol. 159, 2020.
- [27] J. R. Landis and G. G. Koch, “The Measurement of Observer Agreement for Categorical Data,” Biometrics, vol. 33, no. 1, 1977.
- [28] S. Baltes et al., “Guidelines for Empirical Studies in Software Engineering involving Large Language Models,” Empirical Software Engineering, 2026.



Thank you

Questions and discussion

Orthogonal Defect Classification on Defects4J Using an LLM-Driven Scientific Approach

Md. Sakib Hossain · Mohammad Nahiyen Kabir · Hasin Mahtab Alvee